

```

ApplyFoo( float a[] ) :
my_a(a)
{}
};
Once we have the body object ready, we can
call the parallel_for function as follows:
#include "tbb/tbb.h"
void ParallelApplyFoo( float a[],
size_t n ) {
    parallel_for(blocked_range<size_
t>(0,n), ApplyFoo(a));
}

```

INTEL® INTEGRATED PERFORMANCE PRIMITIVES PREVIEW FOR ANDROID*

The Intel IPP preview for Android is a rich library of data and multimedia processing functions for various environments that include not only Android, but also Windows, Linux, and OS X. These functions are highly optimized and use the Intel® Streaming SIMD extensions and Intel® Advanced Vector Extensions (Intel® AVX). Intel IPP caters mainly to Signal processing, Image processing, Matrices, and Cryptography and can be used on any processor that supports SSE2 or newer instruction set. Beacon Mountain bundles a preview of Intel IPP functionality available as a free download. If developers choose to use this library, they can buy an extended version from the Intel web site. The Google Android NDK is a prerequisite before you can use or run a sample program for Intel IPP, because it is called using native code from the C/C++ part of your application. To prepare to run the sample HelloIPP app, we need to create a HelloIPP Android project in the Eclipse IDE, with an activity of the same name. Once this is done, native support can be added to the project by right-clicking on the project name and choosing Android Tools > Add Native Support. This will autogenerate the contents to add support for JNI functions in the application.

To run the application, copy the files from the Beacon Mountain installation directory and enter the source where it is relevant. In our example, the native function retrieves the version of the Intel IPP library and returns the value to the calling Java function. This can be written in HelloIPP.c as follows:

```

#include <jni.h>
#include "ipp_preview.h"
JNIEXPORT void JNICALL
Java_intel_example_HelloIPP_HelloIPPVersion( JNIEnv* env,

```

```

                                jobject thiz,
                                jint
                                tArray *vinfo)
{
    // get a pointer to the Java
    array
                                jint *tmp = (*env)-
>GetIntArrayElements(env, vinfo, 0);
    // call IPP's utility function
    to get library version info
                                const IppLibraryVersion
*lib=ippGetLibVersion();
    // copy this info into the
    array (visible from Java caller)
    tmp[0]=lib->major;
    tmp[1]=lib->minor;
    // release the pointer
                                (*env)-
>ReleaseIntArrayElements(env, vinfo,
tmp, 0);
}

```

The calling function can be called in Java as follows:

```

package intel.example.HelloIPP;
import android.app.Activity;
import android.widget.TextView;
import android.os.Bundle;
public class HelloIPP extends
Activity
{
    public native void
IPPVersion(int[] vinfo);
    @Override
    public void onCreate(Bundle
savedInstanceState)
    {
                                super.
onCreate(savedInstanceState);
        // Call the native function
        which queries the IPP version info
        int[] ippversion=new
int[2];
        IPPVersion(ippversion);
        // Display version info in
        a TextView
                                TextView tv = new
TextView(this);
                                tv.append(" test
major="+ippversion[0]+"
minor="+ippversion[1]);
                                setContentView(tv);
    }
    // Load the native library
    static {

```

```

                                System.loadLibrary("HelloIPP");
                                }
}

```

When you run this application, you should be able to see the Major and Minor version numbers of the Intel IPP library. When you are confident you understand the sample app, you can decide if you want to buy the complete library. Once done, all you need to do is replace the static library and include files in the JNI folder of the application. The bundled sample programs provide advanced examples of how to use the functions for audio-video codecs, computer vision, image processing, speech codes, data compression, etc. There is also a reference guide available that provides a comprehensive list explaining each type of utility function available in the library.

CONCLUSION

Beacon Mountain provides you with all the tools you need to get a jumpstart with Android app development on Intel processor-based mobile phones and tablets. With the help of Beacon Mountain, you no longer need to worry about setting up and configuring the IDE and retrieving everything from multiple sources. Moreover, it provides a rich set of examples to give a taste of how to use high performance libraries like Intel IPP and Intel TBB. ■

Notices

INFORMATION IN THIS DOCUMENT IS PROVIDED IN CONNECTION WITH INTEL PRODUCTS. NO LICENSE, EXPRESS OR IMPLIED, BY ESTOPPEL OR OTHERWISE, TO ANY INTELLECTUAL PROPERTY RIGHTS IS GRANTED BY THIS DOCUMENT. EXCEPT AS PROVIDED IN INTEL'S TERMS AND CONDITIONS OF SALE FOR SUCH PRODUCTS, INTEL ASSUMES NO LIABILITY WHATSOEVER AND INTEL DISCLAIMS ANY EXPRESS OR IMPLIED WARRANTY, RELATING TO SALE AND/OR USE OF INTEL PRODUCTS INCLUDING LIABILITY OR WARRANTIES RELATING TO FITNESS FOR A PARTICULAR PURPOSE, MERCHANTABILITY, OR INFRINGEMENT OF ANY PATENT, COPYRIGHT OR OTHER INTELLECTUAL PROPERTY RIGHT.

Intel, the Intel logo, Atom, Core, and VTune are trademarks of Intel Corporation in the U.S. and other countries.

The Android robot is reproduced or modified from work created and shared by Google and used according to terms described in the Creative Commons 3.0 Attribution License.

*Other names and brands may be claimed as the property of others
Copyright© 2013 Intel Corporation. All rights reserved.